容器基础基础

容器技术

Linux系统中因为大量的软件使用动态链接的方式，所以虽然可以节约很多软件的空间开销，但是依赖问题会是Linux系统中没法逃避的问题

我们现在的主流方式使用容器技术，在我们的Linux系统之上开辟一个隔离环境，有的独立的进程信息，文件系统，网络地址等，但是又不需要像虚拟机一样安装完整的操作系统和硬件层面的虚拟化加速支持

目前主流的容器管理引擎为`Docker`

目前国内的网络依旧无法正常的访问Docker官方站点和Docker Hub镜像仓库

安装和简单使用Docker

这里我们就使用国内的镜像仓库来完成相关的配置和使用

```
dnf update -y

dnf remove container-tools -y
# 红帽自家推出了podman这个工具来替代docker
# 两者之间存在一定的冲突，所以如果想要使用docker，务必移除相关的组件

curl -o /etc/yum.repos.d/docker-ce.repo https://mirrors.aliyun.com/docker-ce/linux/centos/docker-ce.repo
# 下载阿里云提供的docker-ce的仓库配置文件到我们本机的专门的目录位置

dnf install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
# 安装docker相关的组件和依赖

systemctl enable --now docker
# 到这里我们的Docker就安装完毕并可以开始使用了
```

使用Docker运行容器之前，我们需要使用`docker pull`拉取镜像，镜像是制作好的一个运行环境，里面包含了我们需要的所有内容，我们运行容器之前，需要确保镜像能够正常的拉取

```
docker pull ghcr.io/ggml-org/llama.cpp:server
# 直接拉取镜像
```

有时候镜像拉取比较慢，所以有些时候我们可以使用镜像地址或者让别的机器导出拉取好的镜像，本地导入即可

```
docker pull ghcr.1ms.run/ggml-org/llama.cpp:server
# 使用1ms.run提供的免费的镜像访问来下载

curl -o llama.cpp.server.tar https://file.boulderh.top/llama.cpp.server.tar
# 直接下载别人下载好导出的镜像
docker load -i llama.cpp.server.tar
# 从本地文件文件中导入镜像
```

当我们有了容器镜像之后，我们就可以通过`docker run`来设置一些启动相关的参数，运行我们的环境

```
docker run -d --name llama-server -p 8080:8080 -v ~/Qwen3.5-4B:/models ghcr.io/ggml-org/llama.cpp:server -m /models/Qwen3.5-4B-Q4_K_M.gguf -mm /models/mmproj-Qwen3.5-4b-BF16.gguf --host 0.0.0.0 --port 8080 -a Qwen3.5
```